

```

import time
import numpy as np
from typing import Tuple, List

from qiskit.quantum_info import
SparsePauliOp
from qiskit.circuit.library import
QAOAAnsatz
from qiskit.primitives import
StatevectorSampler
from qiskit_aer.primitives import
EstimatorV2 as AerEstimator
from scipy.optimize import minimize

#
=====
=====
# 1. Synthetic Market Data Generation
(unchanged from portfolio_fixed.py)
#
=====
=====
def generate_market_data(seed: int = 42):
    np.random.seed(seed)
    asset_names = ["A1", "A2", "A3", "A4",
"A5", "A6", "A7", "A8", "A9", "A10"]
    mu = np.array([0.08, 0.07, 0.09, 0.03,
0.04, 0.06, 0.02, 0.07, 0.06, 0.01])
    vols = np.array([0.15, 0.16, 0.21,

```

```

0.05, 0.07, 0.18, 0.08, 0.14, 0.10, 0.008])
    N = len(asset_names)
    A = np.random.uniform(0.1, 0.7, size=
(N, N))
    corr = (A + A.T) / 2.0
    np.fill_diagonal(corr, 1.0)
    D = np.diag(vols)
    sigma = D @ corr @ D
    cost = np.array([0.010, 0.012, 0.020,
0.004, 0.006, 0.018, 0.009, 0.015, 0.011,
0.001])
    return mu, sigma, cost, asset_names

#
=====
=====
# 2. QUBO & Ising Mapping Engine (unchanged
from portfolio_fixed.py)
#
=====
=====
class QuantumPortfolioMapper:
    def __init__(self, mu, sigma, cost, q:
float = 0.5, gamma: float = 1.0,
                B: int = 5, P: float =
30.0):
        self.mu, self.sigma, self.cost =
mu, sigma, cost
        self.q, self.gamma, self.B, self.P
= q, gamma, B, P
        self.N = len(mu)
        self.Q_diag, self.Q_off =
self._build_qubo()

```

```

        self.ising_op, self.offset =
self._qubo_to_ising()

    def _build_qubo(self):
        Q_diag = (self.q *
np.diag(self.sigma) - self.mu + self.gamma
* self.cost
                    + self.P * (1.0 - 2.0 *
self.B))
        Q_off = 2.0 * self.q * self.sigma +
2.0 * self.P
        np.fill_diagonal(Q_off, 0)
        return Q_diag, Q_off

    def _qubo_to_ising(self):
        pauli_list = []
        for i in range(self.N):
            h_i = -0.5 * self.Q_diag[i] -
0.25 * np.sum(self.Q_off[i, :])
            z_str = ["I"] * self.N
            z_str[i] = "Z"

pauli_list.append(("".join(reversed(z_str))
, h_i))

        for i in range(self.N):
            for j in range(i + 1, self.N):
                z_str = ["I"] * self.N
                z_str[i] = "Z"
                z_str[j] = "Z"

pauli_list.append(("".join(reversed(z_str))
, 0.25 * self.Q_off[i, j]))
        offset = (0.5 * np.sum(self.Q_diag)
+ 0.25 * np.sum(np.triu(self.Q_off, 1))

```

```

        + self.P * (self.B ** 2))
    return
SparsePauliOp.from_list(pauli_list), offset

    def evaluate_bitstring(self, bitstring:
str, already_asset_order: bool = False) ->
float:
        s = bitstring if
already_asset_order else bitstring[::-1]
        x = np.array([int(b) for b in s])
        return (self.q * (x.T @ self.sigma
@ x) - self.mu.T @ x
                + self.gamma * (self.cost @
x)
                + self.P * ((np.sum(x) -
self.B) ** 2))

    def raw_objective(self, x: np.ndarray)
-> float:
        return self.q * (x.T @ self.sigma @
x) - self.mu.T @ x + self.gamma *
(self.cost @ x)

def to_asset_order(bitstring: str) -> str:
    return bitstring[::-1]

#
=====
=====
# 3. MANUAL QAOA LOOP (replaces
qiskit_algorithms.QAOA wrapper)
#

```

```

=====
=====
class ManualQAOA:
    """
        Minimal, transparent QAOA
    implementation:
        - QAOAAnsatz builds the parameterized
        circuit directly from the cost Hamiltonian.
        - StatevectorEstimator computes  $\langle H \rangle$ 
        for a given parameter set (cost function
        for the classical optimizer).
        - StatevectorSampler draws bitstring
        samples from the final optimized circuit.
        -
        scipy.optimize.minimize(method='COBYLA')
        drives the parameters – same
        optimizer as before, but now we
        control every step, so it's trivial to:
            (a) log convergence,
            (b) swap in CVaR-based sample
            aggregation instead of plain expectation
            value (just change what
            cost_function returns), or
            (c) swap COBYLA for another
            optimizer.
    """

    def __init__(self, cost_hamiltonian:
        SparsePauliOp, reps: int = 2, seed: int =
        42):
        self.hamiltonian = cost_hamiltonian
        self.reps = reps
        self.ansatz =
        QAOAAnsatz(cost_operator=cost_hamiltonian,

```

```

reps=reps)
    # IMPORTANT: the reference
qiskit.primitives.StatevectorEstimator was
measured
    # at ~17s PER CALL for this 10-
qubit circuit in this environment (pure-
Python
    # backend, no circuit
decomposition) -- an 80-iteration
optimization would take
    # ~25 minutes. qiskit-aer's C++
backend runs the same expectation value in
    # ~0.02-0.05s once the ansatz is
decomposed into basis gates (Aer doesn't
    # recognize the QAOAAnsatz's custom
"QAOA" instruction directly). This is a
    # >300x speedup and is what makes
an interactive optimization loop practical.
    self.ansatz_decomposed =
self.ansatz.decompose(reps=3)
    self.estimator = AerEstimator()
    self.sampler = StatevectorSampler()
    self.rng =
np.random.default_rng(seed)
    self.history: List[float] = []

    def _expectation(self, params:
np.ndarray) -> float:
        """Cost function handed to scipy:
<psi(params)| H |psi(params)>."""
        job =
self.estimator.run([(self.ansatz_decomposed
, self.hamiltonian, params)])
        result = job.result()[0]

```

```

        energy = float(result.data.evs)
        self.history.append(energy)
        return energy

    def optimize(self, maxiter: int = 100)
-> dict:
        num_params =
self.ansatz.num_parameters
        x0 = self.rng.uniform(0, np.pi,
size=num_params) # random init in [0, pi]

        t0 = time.time()
        res = minimize(
            self._expectation,
            x0,
            method="COBYLA",
            options={"maxiter": maxiter,
"rhobeg": 0.5},
        )
        elapsed = time.time() - t0

        return {
            "optimal_params": res.x,
            "optimal_energy": res.fun,
            "n_evals": len(self.history),
            "elapsed_sec": elapsed,
            "history": self.history,
        }

    def sample(self, params: np.ndarray,
shots: int = 2048) -> dict:
        """Draw bitstring samples from the
circuit at the given (optimized)
parameters."""

```

```

        bound_circuit =
self.ansatz_decomposed.assign_parameters(pa
rams)
        bound_circuit.measure_all()
        job =
self.sampler.run([bound_circuit],
shots=shots)
        result = job.result()[0]
        counts =
result.data.meas.get_counts()
        return counts

#
=====
=====
# 4. Feasibility repair + classical local
search (unchanged)
#
=====
=====
def repair_feasibility(x: np.ndarray,
mapper: QuantumPortfolioMapper) ->
np.ndarray:
    x = x.copy()
    B = mapper.B
    current = int(np.sum(x))

    if current > B:
        selected = [i for i in
range(mapper.N) if x[i] == 1]
        selected.sort(key=lambda i:
mapper.mu[i] - mapper.gamma *
mapper.cost[i])

```



```

        for i in selected:
            if current <= B:
                break
            x[i] = 0
            current -= 1
        elif current < B:
            unselected = [i for i in
range(mapper.N) if x[i] == 0]
            unselected.sort(key=lambda i:
mapper.mu[i] - mapper.gamma *
mapper.cost[i], reverse=True)
            for i in unselected:
                if current >= B:
                    break
                x[i] = 1
                current += 1

        assert int(np.sum(x)) == B,
"Feasibility repair failed to hit budget
exactly"
        return x

def
classical_local_search_refinement(x_init:
np.ndarray, mapper:
QuantumPortfolioMapper):
    current_x = x_init.copy()
    current_cost =
mapper.evaluate_bitstring("".join(map(str,
current_x))), already_asset_order=True)

    improved = True
    while improved:

```

```

        improved = False
        N = len(current_x)
        for i in range(N):
            for j in range(N):
                if current_x[i] == 1 and
current_x[j] == 0:
                    candidate_x =
current_x.copy()
                    candidate_x[i],
candidate_x[j] = 0, 1
                    cand_cost =
mapper.evaluate_bitstring(
                        "".join(map(str,
candidate_x)), already_asset_order=True
                    )
                    if cand_cost <
current_cost:
                        current_cost =
cand_cost
                        current_x =
candidate_x
                        improved = True
                        break
            if improved:
                break

        return current_x, current_cost

```

```

#
=====
=====
# 5. Main Workflow
#

```

```

=====
=====
if __name__ == "__main__":
    mu, sigma, cost, asset_names =
generate_market_data()
    mapper = QuantumPortfolioMapper(mu,
sigma, cost, q=0.5, gamma=1.0, B=5, P=30.0)

    print(f"Ising Hamiltonian Qubit Count:
{mapper.ising_op.num_qubits}")
    print(f"Number of Pauli terms:
{len(mapper.ising_op)}")
    print(f"Energy Shift Offset:
{mapper.offset:.4f}")

    # --- Manual QAOA optimization ---
    qaoa = ManualQAOA(mapper.ising_op,
reps=2)
    print(f"\nAnsatz parameter count:
{qaoa.ansatz.num_parameters}")

    opt_result = qaoa.optimize(maxiter=200)
    print(f"\n[Manual QAOA] Converged in
{opt_result['elapsed_sec']:.1f}s "
        f"over {opt_result['n_evals']}
cost-function evaluations")
    print(f"[Manual QAOA] Optimal (Ising)
energy:
{opt_result['optimal_energy']:.4f}")
    print(f"[Manual QAOA] Energy history
(first 5): "
        f"{[round(e, 3) for e in
opt_result['history'][:5]]}")
    print(f"[Manual QAOA] Energy history

```

```

(last 5): "
            f"{[round(e, 3) for e in
opt_result['history'][-5:]]}"

    # --- Sample bitstrings from the
    optimized circuit ---
    counts =
    qaoa.sample(opt_result["optimal_params"],
shots=2048)
    top_bitstring = max(counts,
key=counts.get)
    print(f"\n[Manual QAOA] Most frequent
measured bitstring (top of {len(counts)}
unique): "
            f"{top_bitstring} (count=
{counts[top_bitstring]}/2048)")

    # Convert to asset order (undo Qiskit's
    little-endian convention)
    asset_order_str =
    to_asset_order(top_bitstring)
    x_measured = np.array([int(b) for b in
asset_order_str])
    print(f"[Manual QAOA] Bitstring in
asset order: {asset_order_str} "
            f"(selected count =
{int(np.sum(x_measured))}, target B=
{mapper.B}")

    # --- Feasibility repair (guarantees
    Sum(x) == B regardless of QAOA output) ---
    x_feasible =
    repair_feasibility(x_measured, mapper)
    print(f"\n[Repair] Bitstring after

```

```

feasibility repair: {''.join(map(str,
x_feasible)))} "
        f"(selected count =
{int(np.sum(x_feasible)))}")

    # --- Classical local search refinement
    ---
    x_refined, refined_cost =
classical_local_search_refinement(x_feasibl
e, mapper)
    print(f"\n[Hybrid Refinement] Final
bitstring: {''.join(map(str, x_refined)))")
    print(f"[Hybrid Refinement] Selected
count: {int(np.sum(x_refined))} (target B=
{mapper.B})")
    print(f"[Hybrid Refinement] Full
objective incl. penalty:
{refined_cost:.4f}")
    print(f"[Hybrid Refinement] Raw
objective (no penalty):
{mapper.raw_objective(x_refined):.4f}")

    selected_indices = [i for i, bit in
enumerate(x_refined) if bit == 1]
    print("\nSelected Asset Portfolio
(B=5):")
    for idx in selected_indices:
        print(f" - {asset_names[idx]}
(Return: {mu[idx]*100:.1f}%, "
            f"Vol: {np.sqrt(sigma[idx,
idx])*100:.1f}%, Cost:
{cost[idx]*100:.2f}%)")

```

